

Full NLP Final Project Report

TulongText PH

Filipino-English Disaster Relief Post Classification for Humanitarian Triage

An NLP system that classifies disaster and social media posts into humanitarian categories, derives urgency levels, and provides a working Flask web demo for responder-oriented triage. The final deployed model uses a tuned TF-IDF baseline because it achieved the strongest saved evaluation results.

Final selected model: TF-IDF word bigram + calibrated LinearSVC

Current baseline macro F1: 0.7689 | Actionability macro F1: 0.8596

Prepared for NLP Final Project Presentation

Project Summary

TulongText PH is a Filipino-English disaster relief post classifier and urgency triage system. It accepts a social media or disaster-related post and predicts one of eight humanitarian response categories. The system also derives an urgency level and actionability signal, then displays confidence scores, preprocessing output, and top predictions through a Flask-powered web application.

Problem Addressed

During disasters, social media posts can contain urgent rescue requests, medical reports, evacuation information, infrastructure damage, warnings, donation offers, and general updates. Manually reading every post is slow. TulongText PH applies NLP text classification to help responders, LGUs, volunteers, and monitoring teams prioritize posts faster.

Final Model Decision

The final deployed model is the TF-IDF baseline with a calibrated LinearSVC classifier. Although a transformer model was tested, the saved baseline produced stronger evaluation results under local compute constraints. This makes the final system both more reliable for the demo and easier to reproduce.

Main Results

8-class baseline: accuracy 0.7600, macro precision 0.7654, macro recall 0.7733, macro F1 0.7689. Transformer: accuracy 0.6661, macro F1 0.6608. Secondary actionability model: accuracy 0.8683, macro F1 0.8596.

General Objective

Develop an end-to-end NLP application that can classify disaster-related Filipino-English social media posts into humanitarian response categories and present the result in a working browser demo.

Specific Objectives

1. Collect and normalize disaster-response text datasets.
2. Preprocess noisy social media text using NLP cleaning, tokenization, and normalization.
3. Train and compare at least two models: a TF-IDF baseline and a locally fine-tuned transformer.
4. Evaluate models using accuracy, macro precision, macro recall, macro F1-score, weighted F1-score, classification reports, and confusion matrices.
5. Build a Flask API and polished frontend for live prediction.
6. Provide documentation, limitations, and a clear explanation of model choice.

Scope

The main task is 8-class humanitarian category classification. Urgency is derived deterministically from the predicted category. A secondary binary actionability classifier supports triage by predicting whether a post is actionable or non-actionable. The system does not use GPTs, external classifier APIs, or third-party prediction services.

Input

A short social media or disaster-related text post, written in English, Filipino, or Taglish.

Example: "Need rescue sa Brgy. San Isidro, baha na hanggang bubong. May bata at senior na stranded."

Output

The system returns a predicted humanitarian category, urgency level, confidence score, top predictions, cleaned text, token count, and actionability result.

Why NLP Is Appropriate

The input is unstructured natural language. Disaster posts are noisy, multilingual, time-sensitive, and often short. NLP techniques are needed to transform text into features, detect patterns, classify intent, and support rapid triage.

Categories

The final label set contains eight categories: `rescue_or_urgent_needs`, `medical_or_casualties`, `evacuation_or_displacement`, `infrastructure_damage`, `warnings_or_advice`, `donation_or_volunteering`, `general_update`, and `not_humanitarian`.

The project uses multiple crisis-informatics and disaster-response datasets. The final clean subset is based on the stable category dataset, while additional sources were imported and tested as an expansion experiment.

Item	Details
CrisisBench-all-lang/humanitarian	97,929 rows in clean subset
HumAID-events	61,203 rows in clean subset
CrisisLexT26	4,943 rows in clean subset
Clean subset total	164,075 rows
Expanded experiment	Disaster Response Messages and CrisisMMD imported into separate _expanded files
HFAbrar/disaster_response_messages	5,469 unique rows in expanded import
QCRI/CrisisMMD/humanitarian	38 unique rows in expanded import

The expanded dataset was not selected as the final training source because its first 8-class baseline score was lower than the clean subset baseline. This is documented to show honest experimentation rather than assuming more data always improves performance.

A cleaner training file was created to improve label quality and reduce noise. The process removed short low-information posts, plain retweets, ambiguous labels, and normalized duplicates.

Item	Details
general_update	53,559 rows
not_humanitarian	52,225 rows
rescue_or_urgent_needs	14,289 rows
donation_or_volunteering	12,607 rows
infrastructure_damage	11,295 rows
medical_or_casualties	9,584 rows
warnings_or_advice	6,666 rows
evacuation_or_displacement	3,850 rows

Original rows: 220,495. After quality filters: 164,408. Final clean rows after deduplication: 164,075. Removed normalized duplicates: 333.

Text Cleaning

The preprocessing pipeline removes or normalizes noisy social media artifacts such as URLs, mentions, repeated whitespace, and punctuation noise. It preserves meaningful disaster terms, location terms, and category-related keywords.

Normalization

Text is lowercased and cleaned into a consistent form. Hashtags are handled as useful text when possible. Filipino-English and Taglish examples are kept as text features rather than translated through an external API.

Tokenization and Feature Preparation

The system generates cleaned text and tokens for display. For the baseline model, TF-IDF converts text into numerical features. The best baseline uses word bigrams, which capture short phrase patterns such as "need rescue", "power lines", and "evacuation center".

Label Cleaning

Ambiguous source labels were remapped or filtered. For example, `affected_individuals` is not automatically medical unless injury/death is explicit. `response_efforts` is mapped based on aid/response terms. `rescue_volunteering_or_donation_effort` is split using rescue vs donation cues. `terrorism_related` is dropped because it is too broad for the disaster triage taxonomy.

Critical Urgency

rescue_or_urgent_needs and medical_or_casualties. These represent direct rescue requests, stranded people, urgent needs, injuries, casualties, missing people, or medical support.

High Urgency

evacuation_or_displacement and infrastructure_damage. These indicate evacuation centers, displaced residents, shelter needs, damaged roads, power lines, bridges, buildings, landslides, or utility failures.

Moderate Urgency

warnings_or_advice and donation_or_volunteering. These include alerts, advisories, caution messages, relief goods, volunteer calls, food, water, or supply offers.

Low Urgency

general_update and not_humanitarian. These include general disaster updates, support messages, non-urgent information, unrelated posts, promotional posts, or commercial content.

Urgency Derivation

Urgency is not separately learned by the main model. It is derived from the predicted category using a deterministic mapping. This makes the triage logic transparent and easy to explain during the final presentation.

The final selected 8-class model is the TF-IDF baseline with calibrated LinearSVC. It was chosen because it achieved the strongest saved evaluation results and is fast enough for a stable live demo.

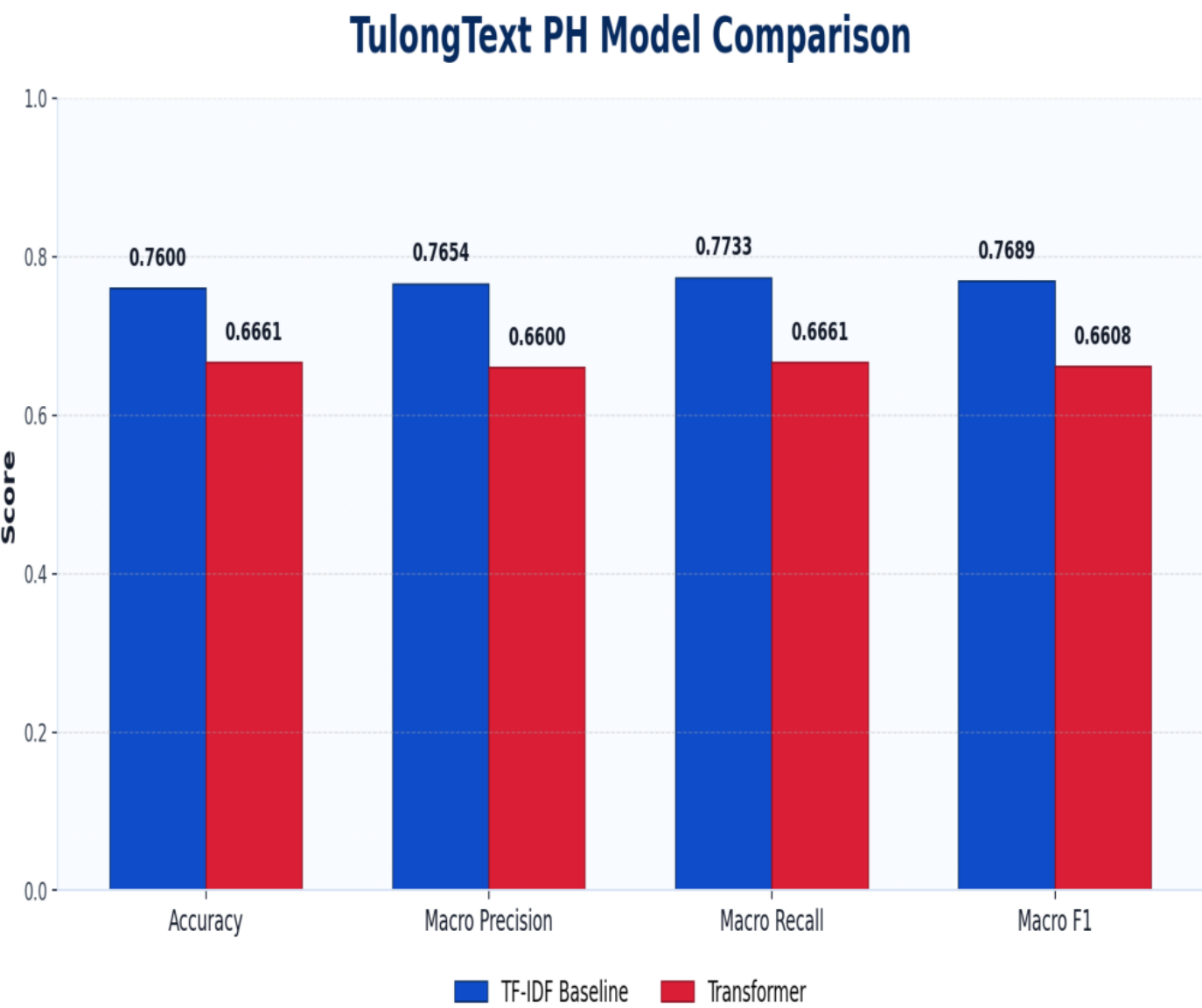
Item	Details
Selected model	TF-IDF word bigram + calibrated LinearSVC
Accuracy	0.7600
Macro Precision	0.7654
Macro Recall	0.7733
Macro F1-score	0.7689
Weighted F1-score	0.7576

The baseline outperformed the saved transformer on the current evaluation files. It is therefore the deployed model for the final app demo.

The project also includes a locally fine-tuned multilingual transformer and a secondary binary actionability classifier. The transformer was tested honestly, but local compute limits and label noise made it weaker than the baseline in the saved run.

Item	Details
Model	DistilBERT multilingual fine-tuned locally
Accuracy	0.6661
Macro Precision	0.6600
Macro Recall	0.6661
Macro F1-score	0.6608
Selected model	Word + character TF-IDF with SGD classifier
Accuracy	0.8683
Macro Precision	0.8535
Macro Recall	0.8690
Macro F1-score	0.8596
Weighted F1-score	0.8697

The actionability model is not a replacement for the 8-class classifier. It is an additional triage signal that helps distinguish actionable posts from non-actionable posts.



Final selected model: TF-IDF baseline, because it achieved stronger saved evaluation results on the 8-class task.

The bar chart shows that the TF-IDF baseline achieved higher scores than the saved transformer across accuracy, macro precision, macro recall, and macro F1-score. This supports the final decision to deploy the baseline model for the live demonstration.

Frontend

The website is a polished humanitarian AI dashboard. It includes TulongText PH branding, a hero section, input and analysis cards, demo examples, prediction categories, urgency badges, confidence bars, cleaned text, and top prediction bars. Demo examples automatically fill the input field and scroll back to the analysis area.

Backend

The backend is built with Flask. It loads the trained baseline, transformer, and actionability models when the server starts. It now also serves the frontend files directly, so users can open the app from the Flask URL instead of manually opening index.html.

API Endpoints

GET /api/health checks server and model status. GET /api/models lists available models and labels. POST /api/predict predicts a single post. POST /api/batch_predict predicts multiple posts.

Demo URL

The app can be opened at <http://127.0.0.1:5000> or the local network address shown by Flask, such as <http://192.168.1.36:5000>, after running `python app.py`.

Step 1: User Input

The user enters a post such as: "Power lines are down along Mabini Street after the typhoon. Avoid the area."

Step 2: Preprocessing

The system cleans the text, normalizes it, tokenizes it, and prepares it for model prediction.

Step 3: Category Prediction

The selected classifier predicts the most likely category, such as `infrastructure_damage`.

Step 4: Urgency Mapping

The app maps `infrastructure_damage` to high urgency because damaged utilities and roads can affect safety and response operations.

Step 5: Output Display

The frontend displays the predicted category, urgency badge, confidence score, top predictions, cleaned text, token count, and actionability result.

Observed Difficulty

The hardest labels are `general_update`, `warnings_or_advice`, `donation_or_volunteering`, and `rescue_or_urgent_needs` because disaster posts often contain overlapping wording. For example, a post asking for food, water, and rescue may contain both urgent-need and donation-related language.

Why 90 Percent Is Difficult

A true 90 percent score is unlikely for the honest 8-class task because the dataset is noisy, multilingual, imbalanced, and merged from multiple corpora. Some labels describe concepts that overlap naturally in real crisis communication.

Model Selection Insight

The baseline performed better than the saved transformer because TF-IDF was strong for keyword-heavy disaster categories, while the transformer required longer stable training and cleaner labels to outperform it. This is a valid experimental result and supports evidence-based model selection.

Mitigation

The project improved performance by creating a clean training subset, removing duplicates, auditing labels, adding additional baseline candidates, and adding a secondary actionability model for a broader triage signal.

Limitations

The system is not a replacement for trained responders. It may misclassify ambiguous posts, sarcasm, mixed requests, location-only updates, or very short text. It does not verify whether a social media post is real. It also does not extract exact geolocation or route information.

Compute Constraints

Transformer fine-tuning was limited by local hardware and long training time. A longer run was attempted, but laptop sleep interrupted the process before final saving. The final app therefore uses the stronger saved baseline model.

Future Improvements

Future work can include checkpointed transformer training, XLM-RoBERTa experiments, manual label review, location extraction, named entity recognition for barangays/cities, near-duplicate detection, active learning, confidence calibration, and a responder dashboard for filtering urgent posts.

Ethical Considerations

The system should assist human triage rather than automatically decide response priority. Predictions should be interpreted with caution, especially when posts involve possible injuries, missing people, or life-threatening conditions.

Conclusion

TulongText PH demonstrates an end-to-end NLP workflow: dataset preparation, preprocessing, label mapping, baseline and transformer comparison, evaluation, actionability modeling, Flask API development, and browser-based demo deployment. The final model choice is evidence-based: the TF-IDF calibrated LinearSVC baseline achieved the strongest saved results and provides a stable, fast demo for disaster-response triage.

Rubric Alignment

The project includes a clear NLP problem, relevant datasets, preprocessing, model development, baseline comparison, evaluation metrics, error analysis, documented limitations, and a working software demo. This directly supports a high-scoring final project presentation.

References and Dataset Sources

CrisisLexT26 disaster tweets. HumAID-events from QCRI. CrisisBench-all-lang from QCRI. SEACrowd Typhoon Yolanda tweets for local relevance support. HFAbrar Disaster Response Messages for optional expansion. QCRI CrisisMMD for optional expansion. Hugging Face datasets were used for dataset acquisition and documentation; no external prediction API was used.